Ciência de Dados e Inteligência Artificial
Disciplina: Computação Escalável
Professor: Larissa C. Shimomura

Universidade Estadual de Londrina

Trabalho do Semestre: Processamento Paralelo e Distribuído de Dados

1. VISÃO GERAL E REQUISITOS DO PROJETO

O objetivo deste trabalho é implementar dois algoritmos de mineração ou processamento de dados em ambiente paralelo ou distribuído e analisar a escalabilidade das soluções conforme o volume de dados aumenta.

1.1 Divisão dos Grupos

- 5 grupos de 4 pessoas.
- 1 grupo de 3 pessoas.

1.2 Escolha da Tarefa e Algoritmos

- Escolher 1 tipo de tarefa da Ciência de Dados (ex.: clustering, classificação, regressão, associação, etc.).
- Implementar 2 algoritmos diferentes para a tarefa escolhida.
- **Divisão interna:** Cada dupla do grupo fica responsável pela implementação de 1 dos algoritmos.

Outras tarefas que podem ser consideradas também incluem tarefas de processamento de dados em geral como, algoritmos para realizar junção entre tabelas, processamento de grafos e redes ou ordenação de grandes volumes de dados. Particularmente, recomendo escolher algoritmos em que vocês tenham uma noção ou algum conhecimento prévio sobre a implementação devido ao curto tempo até a entrega final.

2. INSTRUÇÕES ESPECÍFICAS PARA O GRUPO DE 3 PESSOAS

- **Escolha do Algoritmo:** Escolher 1 algoritmo de Ciência de Dados.
- **Implementação:** Implementar esse algoritmo aplicando 2 decomposições diferentes de dados ou tarefas (ex.: dois métodos distintos de particionamento de dados).
- **Padronização:** Ambas as decomposições devem rodar sob a mesma interface de execução e gerar o mesmo formato de saída (*output*).

2.1 Linguagem e Tecnologias

- A implementação deve utilizar Java (utilizando `Threads`, `ExecutorService` e `ForkJoin` vistos em aula) ou Scala sobre o Apache Spark.
- Ambos os algoritmos do grupo devem obrigatoriamente usar a mesma linguagem.

No caso de utilizar Apache Spark, pode ser utilizado docker containers para montar a estrutura de Master/Worker no seu ambiente local. Também é possível rodar o Apache Spark no modo local (como local[*] ou local[k]), ele executa na mesma JVM e utiliza threads locais de forma bastante análoga ao que fazemos em Java com ExecutorService ou ForkJoin, mas com a abstração dos RDDs e DataFrames do Spark (veremos estas diferenças durante as aulas). O setup utilizado deve ser descrito no README.

2.2 Interface e Padronização

- Os dois algoritmos devem rodar sob a mesma interface de execução e gerar o mesmo formato de saída (*output*) para garantir uma comparação justa.

2.3 Datasets e Teste de Escalabilidade

- Selecionar ou gerar 2 datasets distintos para avaliar o tempo de execução e a escalabilidade dos algoritmos.
- **Origem dos dados:** Podem ser datasets reais ou gerados sinteticamente (caso sejam sintéticos, devem utilizar distribuições de dados diferentes).
- **Tamanho dos datasets:** 10.000 a 150.000 pontos (ou pelo menos na casa dos 100k).
- **Incremento:** Dado o dataset, deve-se aumentar de pouco a pouco (10k a 10k pontos, por exemplo) o número de dados e marcar o tempo de execução do programa a cada ponto. Nota: para obter uma noção da escalabilidade dos algoritmos implementados, deve-se marcar pelo menos 5 pontos no gráfico.
- **Cuidados com a medição:** Cuidado ao marcar o tempo para não incluir inicializações não realizadas pelo programa implementado (por exemplo, tempo de leitura inicial ou inicialização de framework).

2.4 Entregável (Repositório GitHub)

Cada grupo deve criar um repositório no GitHub contendo:

- Códigos-fonte dos dois algoritmos.
- Scripts de execução dos testes de escalabilidade.
- `README.md` completo com instruções claras de como compilar e rodar os testes.

Estas instruções devem estar claras o suficiente para eu pode reproduzir os testes e as implementações.

3. AVALIAÇÃO DE ESCALABILIDADE E EXPERIMENTOS

O objetivo principal desta etapa é avaliar o comportamento sobre a escalabilidade dos algoritmos implementados em ambiente paralelo ou distribuído.

3.1 Construção dos Gráficos e Visualização

Para cada dataset, os grupos devem gerar representações visuais (como gráficos de linha ou outros tipos adequados de gráficos) onde:

- O **eixo X** representa o tamanho do dataset (aumentando gradualmente, por exemplo, de 10k em 10k pontos, desde 10.000 até 150.000 pontos).
- O **eixo Y** representa o tempo de execução (em milissegundos ou segundos).
- Cada gráfico deve conter **duas séries de dados** (por exemplo, duas linhas ou abordagens visuais equivalentes), cada uma representando um dos algoritmos implementados pelo grupo. Os grupos têm liberdade para escolher o tipo de gráfico, desde que seja fácil e claro de visualizar a escalabilidade dos algoritmos.

A Figura 1 ilustra um exemplo de gráfico de escalabilidade esperado no relatório final.

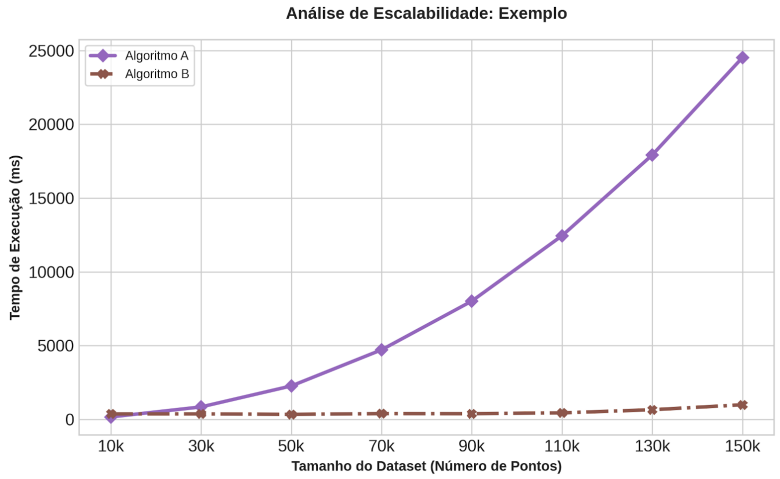


Figura 1: Exemplo de comportamento de escalabilidade (Tamanho do Dataset vs. Tempo de Execução).

Para garantir uma avaliação mais justa possível, todos os testes finais devem obedecer às seguintes diretrizes estritas de ambiente:

- **Mesma Máquina:** Todos os testes comparativos de ambos os algoritmos devem ser executados na mesma máquina física. Recomenda-se distribuir as tarefas sobre execução dos testes e escrita do relatório entre os membros do grupo da melhor forma possível.
- **Recursos Computacionais Controlados:** Utilize o mesmo número de núcleos (*cores*) / *threads* e o mesmo limite de memória disponível para a JVM em ambas as implementações.

- **Isolamento de Processos:** Evite a execução de outras aplicações no computador durante os testes, impedindo que processos em segundo plano influenciem ou distorçam o tempo de execução. Observem que será necessário basicamente parar o uso do computador durante os testes, portanto se programem de acordo.

Avaliações adicionais opcionais: A avaliação sobre o tempo de execução descrita acima é obrigatória na entrega. Avaliações adicionais opcionais incluem, tempo de execução de acordo com o número de threads/workers para um dataset de tamanho fixo ou ao invés de medir tempo de execução, medir o consumo de memória total do algoritmo durante sua execução.

4. SUGESTÕES DE TAREFAS E ALGORITMOS

Os grupos têm liberdade para propor outras tarefas e algoritmos além das sugestões abaixo, porém dois grupos **não podem escolher a mesma combinação de algoritmos**. A confirmação do tema será feita por ordem de envio/registro. Para auxiliar todos na comunicação e escolha de algoritmos, utilizem a Planilha disponibilizada no google classroom.

Tabela 1: Tarefas de Ciência de Dados e Algoritmos Recomendados

Tarefa	**Algoritmos / Abordagem**	**Observações**
Clustering	k-means e DBSCAN	Grupos de 4 pessoas
Clustering 2	Hierárquico e k-Medoids	Grupos de 4 pessoas
Mineração de regras de associação	Apriori e FP-Growth	Grupos de 4 pessoas
Busca k-NN	LSH e k-NN Força Bruta	Requer particionamento eficiente para o força bruta
Mineração de texto	TF-IDF e CountVectorizer	Grupos de 4 pessoas
Supervisionado	Random Forest	Exclusivo para o grupo de 3 pessoas (duas decomposições)

5. CRONOGRAMA DE ENTREGAS E APRESENTAÇÃO

- **Até 25/10 — Formulário Inicial (Google Forms no Classroom):** Preenchimento com: integrantes do grupo, tarefa escolhida, algoritmos a serem implementados, divisão das duplas responsáveis por cada algoritmo e link do repositório no GitHub.
- **Até 12/11 — Entrega do Relatório Final:** Submissão do relatório final através do Google Classroom, contendo a explicação sobre o algoritmo implementado e as decisões tomadas para o paralelismo/distribuição, informações sobre os experimentos e análises de escalabilidade, discussão dos resultados e sugestões de melhorias/trabalhos futuros. O template para o relatório final está disponível no Google Classroom.

- **Dia 13/11 — Apresentação Final:** Apresentação de aproximadamente 20 minutos por grupo, detalhando os algoritmos implementados, decisões de projeto e resultados obtidos.
- **Observação:** A P2 será realizada no dia 27/11.

O processo de avaliação do trabalho considera a entrega pontual dos artefatos e a participação ativa na apresentação final.

Política de Atrasos no Relatório Para cada dia de atraso na entrega do relatório final, será aplicado um desconto de 10% sobre a nota obtida na avaliação do documento. Por exemplo, caso um grupo submeta o relatório final no dia 13 de novembro em vez do prazo limite de 12 de novembro, a nota máxima possível para o relatório passará a ser 9 em vez de 10.

Apresentação Final A apresentação final dos trabalhos é obrigatória e deve ser realizada exclusivamente no dia 13/11, que corresponde ao último dia de aula, visto que não há possibilidade de realizá-la em uma segunda data. Caso o grupo não apresentar o trabalho, o grupo não receberá nota de apresentação final (veja a seção de critérios de avaliação abaixo).

Ressalta-se que, mesmo em caso de atraso na entrega do relatório, o grupo ainda poderá e deverá realizar a apresentação no dia 13/11. A avaliação da apresentação é coletiva, exigindo a participação de todos os membros do grupo. Caso algum integrante não possa comparecer por motivos pessoais ou de saúde, a situação deve ser comunicada por e-mail contudo, o estudante ausente ainda terá a obrigação de colaborar com o grupo (desenvolvendo os slides ou estruturando a apresentação para que um colega o substitua no momento da apresentação).

5.1 Política de Uso de Inteligência Artificial

- O uso de IA **não é permitido** para a implementação do algoritmo distribuído ou paralelo.
- É permitido utilizar IA **apenas** para implementar funções de leitura inicial do arquivo ou de escrita final dos resultados. Qualquer função que faz parte do método implementado não é permitido o uso de IA.
- É permitido utilizar IA para auxiliar no setup de bibliotecas e problemas com versionamento ou containers (dockers) no caso do uso do Apache Spark.
- Também é permitido utilizar IA para gerar os scripts para automatizar os testes de escalabilidade e realizar os gráficos (*plots*) dos resultados finais.
- O uso de IA será verificado de acordo com o repositório da equipe. Favor incluir no **README** e no **relatório** se foi utilizada ou não, deixando de forma explícita para o que foi empregada e quais arquivos/funções do repositório foram afetados.

5.2 Critérios de Avaliação e Distribuição da Nota

A avaliação do trabalho será dividida em três componentes principais: **Implementação e Repositório (40%)**, **Relatório Técnico (40%)** e **Apresentação Final (20%)**, somando o total de 10,0 pontos.

5.3 Detalhamento dos Critérios

1. **Implementação e Repositório GitHub**
    - **Correção e Paralelismo/Distribuição:** Os algoritmos foram implementados corretamente na linguagem escolhida (Java com Threads/ExecutorService/ForkJoin ou Scala/Spark), explorando adequadamente o paralelismo/distribuição de dados sem erros de concorrência ou sincronização.
    - **Interface e medição de tempo:** Ambos os algoritmos utilizam a mesma interface de execução, marca o tempo correto de execução e permitem uma comparação direta e justa.
    - **Organização do Repositório e reproducibilidade:** README com instruções claras para compilação/execução, scripts de automação dos testes presentes (se é possível reproduzir a execução) e declaração explícita do uso (ou não) de IA generativa.
2. **Relatório Técnico e Análise de Escalabilidade**
    - **Descrição Teórica e Decisões de Projeto:** Explicação clara do algoritmo, da estratégia de partição/decomposição adotada e das decisões de arquitetura paralela.
    - **Metodologia Experimental e Datasets (1,0 pte):** Utilização correta dos 2 datasets (tamanho no intervalo recomendado de 10k a 150k pontos) com incrementos graduais.
    - **Análise de Escalabilidade e discussão:** Apresentação dos gráficos comparativos (Tamanho do Dataset vs. Tempo de Execução) para cada dataset e discussão aprofundada dos resultados obtidos, incluindo limitações da solução e sugestões de melhorias.
3. **Apresentação Final**
    - **Clareza e Estrutura da Apresentação** Exposição organizada dentro do tempo limite de 20 minutos, utilizando recursos visuais adequados.
    - **Domínio Técnico do Grupo** Capacidade de explicar as decisões de código, a decomposição paralela e interpretar os resultados dos gráficos.
    - **Participação e Resposta a Pergunta**

Observação: A nota total do trabalho corresponde a 20% da nota da disciplina que é composta por: P1 - 40%, P2 - 40% e entrega deste trabalho - 20%